

TCS NQT — Easy Coding Questions, Compiled from Past Papers (2020-2026)

This is a deep compilation, pulled together from multiple placement-prep archives (GitHub coding sheets, GeeksforGeeks/PrepInsta/Testbook TCS NQT banks, and candidate exam-day reports) plus repeated "actually asked in the real exam" problems. Everything here sits at the **easy** end — the kind of thing you should be able to write correctly in 5-10 minutes once you've drilled it a few times.

Since TCS reuses patterns far more than exact wording, treat every line below as "practice this *type*," not "memorize this exact problem."

A. Array — Easy

1. Find the smallest element in an array
2. Find the largest element in an array
3. Find the second-largest and second-smallest element
4. Reverse an array in place
5. Count the frequency of each distinct element
6. Calculate the sum of all elements
7. Find the average of all elements
8. Find the median of an array
9. Rotate an array left/right by K positions
10. Remove duplicates from a **sorted** array
11. Remove duplicates from an **unsorted** array
12. Insert a new element at a given position
13. Find all elements that repeat vs. all elements that appear only once
14. Find "symmetric pairs" — pairs (a,b) and (b,a) both present
15. Search for a given element (linear/binary search)
16. Check if one array is a subset of another
17. Find the equilibrium index (left-sum equals right-sum at that index)
18. **Push all zeros to the end while keeping order** (the well-known "chocolate factory" story version: an array of chocolate packets where $\emptyset$ = empty packet, shown below) — this exact one has been reported multiple times across recent cycles.

Sample of #18 as it's been phrased in real papers: $N=8$, $arr = [4,5,0,1,9,0,5,0] \rightarrow output\ 4\ 5\ 1\ 9\ 5\ 0\ 0\ 0$. Two-pointer, in-place, $O(n)$.

B. String — Easy

1. Check if a string is a palindrome
 2. Count vowels, consonants, and spaces in a string
 3. Find the ASCII value of a character
 4. Remove all vowels from a string
 5. Remove all spaces from a string
 6. Strip everything except alphabetic characters
 7. Reverse a string
 8. Remove brackets from an algebraic expression
 9. Sum up all the numbers embedded in a string
 10. Capitalize the first and last letter of every word
 11. Frequency count of each character
 12. Find the first non-repeating character
 13. Check if two strings are anagrams of each other
 14. Find the character that occurs the maximum number of times
 15. Remove duplicate characters from a string
 16. Print out just the duplicate characters
 17. Remove all characters of string A that also appear in string B
 18. Shift every letter to the next one in the alphabet ($a \rightarrow b, z \rightarrow a$)
 19. Find the longest word in a sentence
 20. Sort the characters of a string
 21. Count the number of words in a sentence
 22. Reverse the order of words in a sentence (not the letters)
 23. Find the starting index of a substring inside a string
 24. Find the **longest common prefix** among an array of strings
($(["flower", "flow", "flight"] \rightarrow "fl")$)
 25. **Backspace string compare** — two typed strings use $\textcircled{\#}$ as a delete key; check whether both strings resolve to the same final text. This one shows up almost every cycle in some phrasing.
-

C. Numeric / Math — Easy

1. Check if a number is even or odd
2. Check if a number is positive, negative, or zero
3. Find the greatest of two numbers / three numbers
4. Sum of the first N natural numbers
5. Sum of an Arithmetic Progression (AP) / Geometric Progression (GP) series

6. Check if a number is a palindrome
 7. Check if a number is prime; list primes in a range
 8. Check if a number is an **Armstrong number** ($153 \rightarrow 1^3 + 5^3 + 3^3 = 153$)
 9. Check if a number is a **perfect number** (sum of divisors = the number)
 10. Check if a number is a **strong number** (sum of factorials of digits = the number)
 11. Check if a number is **Automorphic** (its square ends in the number itself, e.g. $25^2 = 625$)
 12. Check if a number is a **Harshad number** (divisible by the sum of its own digits)
 13. Check if a number is **abundant** (sum of divisors > the number)
 14. Check leap year
 15. Reverse the digits of a number
 16. Find the maximum and minimum digit within a number
 17. Print the Fibonacci sequence up to N terms
 18. Find the factorial of a number
 19. Compute the power of a number (a^b) without using a built-in power function
 20. Print all factors, and separately all **prime factors**, of a number
 21. GCD and LCM of two numbers
 22. Sum of digits of a number
 23. Sum of all numbers within a given range
 24. Check if a number can be expressed as the sum of two primes
 25. Find the roots of a quadratic equation
 26. **Toggle all bits after (and including) the most significant set bit** of a number's binary form — a real "digital logic" story question TCS has asked ($10 \rightarrow 1010 \rightarrow$ toggled $0101 \rightarrow$ output 5).
 27. **Count handshakes:** if N people each shake hands with every other person exactly once, how many handshakes happen in total? (Answer: $N \times (N-1) / 2$ — this exact "everyone got infected" story problem has appeared in real papers.)
 28. Real-world "tiered rate" word problems dressed up as: parking fee calculator, gym membership cost, purchase discount calculator, transaction monitoring limit — all of these reduce to plain **if/else on numeric ranges**; don't be fooled by the story.
-

D. Number System — Easy

1. Binary $\rightarrow$ Decimal
2. Decimal $\rightarrow$ Binary
3. Binary $\rightarrow$ Octal
4. Octal $\rightarrow$ Binary
5. Decimal $\rightarrow$ Octal

6. Octal → Decimal
 7. Convert a number into words (e.g. 125 → "One Hundred Twenty Five")
-

E. Sorting — Easy (be able to code these from scratch, no library calls)

1. Bubble sort
2. Selection sort
3. Insertion sort
4. Quick sort (partition logic)
5. Merge sort (merge-two-sorted-halves logic)

You won't usually be asked to "implement bubble sort" directly as the whole problem — but plenty of easy problems above (e.g. sort characters in a string, rearrange array in increasing-decreasing order, sort by frequency) expect you to either use a built-in sort correctly or write simple sort logic when the built-in doesn't fit the required order.

F. Real exam-reported easy questions (verbatim style, recent cycles)

These are close to how they were actually phrased on real test days, based on candidate write-ups:

- *"Check whether a number leaves a remainder of 3 when divided by 10, and print True/False."* — trivial once decoded, but the wording is deliberately indirect. Read modulo-style problems twice before coding.
 - *"Count pairs of numbers in a stack (or array) where the absolute difference equals a given K."*
 - *"A chocolate factory conveyor belt array — push empty packets (0s) to the end."* (Array #18 above)
 - *"Toggle bits after the MSB of a binary number."* (Numeric #26 above)
 - *"Count handshakes among N people in a room."* (Numeric #27 above)
 - *"Two kids typing on a typewriter with a backspace key (#) — do their final strings match?"* (String #25 above)
-

How to actually drill this in 3 weeks

- Don't try to memorize solutions to 100 problems. Instead, get **fluent in the 5 techniques** that these questions keep reusing: single-pass array traversal (min/max/second-max/frequency), two-pointer (zeros-to-end, palindrome, backspace-compare), digit extraction ($(\%10)$, $(/10)$), basic recursion (factorial, Fibonacci, GCD), and if/else range logic (the "story problems").
- Time yourself once you can solve a category cold: under 10 min for anything in Array/String/Numeric-easy above, since the real test gives you far less breathing room

across 2 problems in 60-90 minutes.

- Re-read the problem statement twice before typing — several of TCS's "easy" questions (remainder-of-3, toggle-bits, handshake) are simple logic wrapped in unnecessarily wordy stories specifically to slow you down under time pressure.